ORBIT · PYTHON · PAPER 07

第 7 套

60 题 · 完整题目与答案

单选 30 多选 15 填空 / 简答 15

本套分节

一、单项选择题（30） 二、不定项选择题（15） 三、填空题（5） 四、程序代码阅读题（5） 五、程序设计题（5）

答案与解析紧随每题；本文件用于复习核对，建议先独立作答。

一、单项选择题

01 • 单项选择

以下代码的输出是什么？

```
def func(x, lst=[]):
    lst.append(x)
    return lst
a = func(1)
b = func(2, [])
c = func(3)
print(len(a))
```

A. 1
B. 2
C. 3
D. 报错

答案 B. 2

解析

02 • 单项选择

关于列表的+=操作，以下说法正确的是：

```
lst1 = [1, 2]
lst2 = lst1
lst1 += [3, 4]
```

A. lst2不受影响
B. lst2也变成[1, 2, 3, 4]
C. 创建了新列表
D. 报错

答案 B. lst2也变成[1, 2, 3, 4]

解析

03 • 单项选择

以下代码的输出是什么？

```
t = (1, 2, [3, 4])
t[2] += [5]
```

A. 正常执行，t变为(1, 2, [3, 4, 5])
B. 抛出TypeError
C. t不变
D. A和B都对（先修改后报错）

答案 D. A和B都对（先修改后报错）

解析

04 • 单项选择

关于字典的get()方法和直接索引，以下说法正确的是：

A. 两者完全相同

B. get()不存在时返回None，索引会抛出KeyError
C. get()更慢
D. 索引更安全

答案 B. get()不存在时返回None，索引会抛出KeyError

解析

05 • 单项选择

以下代码的输出是什么？

```
d = {'a': 1}
d2 = d
d2['b'] = 2
print(len(d))
```

A. 1
B. 2
C. 0
D. 报错

答案 B. 2

解析

06 • 单项选择

关于列表的sort()和sorted()，以下说法正确的是：

A. sort()返回新列表
B. sorted()修改原列表
C. sort()修改原列表，sorted()返回新列表
D. 两者完全相同

答案 C. sort()修改原列表，sorted()返回新列表

解析

07 • 单项选择

以下代码的输出是什么？

```
lst = [3, 1, 2]
result = sorted(lst)
print(lst)
```

A. [1, 2, 3]
B. [3, 1, 2]
C. None
D. 报错

答案 B. [3, 1, 2]

解析

08 • 单项选择

关于集合的可变性，以下说法正确的是：

A. 集合元素必须是可变对象
B. 集合元素必须是不可变对象

C. 集合元素可以是任何对象

D. 集合不能包含数字

答案 B. 集合元素必须是不可变对象

解析

09 • 单项选择

以下代码会发生什么？

```
s = {[1, 2], [3, 4]}
```

A. 正常创建集合

B. 抛出TypeError

C. 创建空集合

D. 创建列表

答案 B. 抛出TypeError

解析

10 • 单项选择

关于函数参数的传递，以下说法正确的是：

```
def modify(x):
    x = x + [4]
    return x
lst = [1, 2, 3]
result = modify(lst)
```

A. lst被修改

B. lst不被修改

C. result是None

D. 报错

答案 B. lst不被修改

解析

11 • 单项选择

以下代码的输出是什么？

```
def append_and_return(lst):
    lst.append(4)
    return lst
original = [1, 2, 3]
new = append_and_return(original)
print(original is new)
```

A. True

B. False

C. None

D. 报错

答案 A. True

解析

12 • 单项选择

关于字符串的拼接，以下哪种方式效率最高？

A. 使用+循环拼接
B. 使用join()方法
C. 使用+=
D. 都一样

答案 B. 使用join()方法

解析

13 • 单项选择

以下代码的输出是什么？

```
s1 = "hello"
s2 = s1
s1 += " world"
print(s2)
```

A. "hello world"
B. "hello"
C. ""
D. 报错

答案 B. "hello"

解析

14 • 单项选择

关于列表的clear()方法，以下说法正确的是：

A. 返回新的空列表
B. 清空原列表
C. 删除列表对象
D. 不修改列表

答案 B. 清空原列表

解析

15 • 单项选择

以下代码的输出是什么？

```
lst1 = [1, 2, 3]
lst2 = lst1
lst1.clear()
print(lst2)
```

A. [1, 2, 3]
B. []
C. None
D. 报错

答案 B. []

解析

16 • 单项选择

Pandas中，以下哪个操作会创建视图而不是副本？

A. df'col1', 'col2'
B. df.loc[:, 'col1']
C. df.iloc[0:5]
D. 不确定，取决于具体情况

答案 D. 不确定，取决于具体情况

解析

17 • 单项选择

以下代码的输出是什么？

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2, 3]})
view = df['A']
view[0] = 100
print(df['A'][0])
```

A. 1
B. 100
C. 可能是1或100
D. 报错

答案 C. 可能是1或100

解析

18 • 单项选择

关于元组的性能优势，以下说法正确的是：

A. 元组比列表占用更少内存
B. 元组的访问速度更快
C. 元组可以作为字典的键
D. 以上都正确

答案 D. 以上都正确

解析

19 • 单项选择

以下代码的输出是什么？

```
t1 = (1, 2, 3)
t2 = (1, 2, 3)
print(t1 is t2)
```

A. True
B. False
C. 可能是True或False
D. 报错

答案 C. 可能是True或False

解析

20 • 单项选择

关于列表的reverse()和reversed()，以下说法正确的是：

A. reverse()返回新列表
B. reversed()修改原列表
C. reverse()修改原列表，reversed()返回迭代器
D. 两者完全相同

答案 C. reverse()修改原列表，reversed()返回迭代器

解析

21 • 单项选择

以下代码的输出是什么？

```
lst = [1, 2, 3]
rev = reversed(lst)
print(type(rev))
```

A. <class 'list'>
B. <class 'list_reverseiterator'>
C. <class 'reversed'>
D. <class 'iterator'>

答案 B. <class 'list_reverseiterator'>

解析

22 • 单项选择

关于字典的keys()、values()、items()，以下说法正确的是：

A. 返回列表
B. 返回视图对象
C. 返回元组
D. 返回集合

答案 B. 返回视图对象

解析

23 • 单项选择

以下代码的输出是什么？

```
d = {'a': 1, 'b': 2}
keys = d.keys()
d['c'] = 3
print(len(keys))
```

A. 2
B. 3
C. 0
D. 报错

答案 B. 3

解析

24 • 单项选择

关于列表的*操作符，以下说法正确的是：

```
lst = [[]] * 3
lst[0].append(1)
```

A. 只有lst[0]被修改
B. 所有子列表都被修改
C. 报错
D. 创建深拷贝

答案 B. 所有子列表都被修改

解析

25 • 单项选择

以下代码的输出是什么？

```
matrix = [[0] * 2] * 3
matrix[0][0] = 1
print(matrix)
```

A. 1, 0], [0, 0], [0, 0
B. 1, 0], [1, 0], [1, 0
C. 0, 0], [0, 0], [0, 0
D. 报错

答案 B. 1, 0], [1, 0], [1, 0

解析

26 • 单项选择

关于函数的默认参数求值时机，以下说法正确的是：

A. 每次调用时求值
B. 函数定义时求值一次
C. 第一次调用时求值
D. 随机求值

答案 B. 函数定义时求值一次

解析

27 • 单项选择

以下代码的输出是什么？

```
import time
def func(t=time.time()):
    return t
a = func()
time.sleep(1)
b = func()
print(a == b)
```

A. True
B. False
C. 可能是True或False
D. 报错

答案 A. True

解析

28 • 单项选择

关于列表的pop()方法，以下说法正确的是：

A. 不修改原列表
B. 修改原列表并返回删除的元素
C. 只能删除最后一个元素
D. 返回新列表

答案 B. 修改原列表并返回删除的元素

解析

29 • 单项选择

以下代码的输出是什么？

```
lst = [1, 2, 3, 4, 5]
x = lst.pop(2)
print(x, len(lst))
```

A. 3 4
B. 2 4
C. 3 5
D. 报错

答案 A. 3 4

解析

30 • 单项选择

关于Pandas的inplace参数，以下说法正确的是：

A. inplace=True修改原对象
B. inplace=False返回新对象
C. 默认值通常是False
D. 以上都正确

答案 D. 以上都正确

解析

二、不定项选择题

31 • 不定项选择

以下哪些是可变对象？

A. list
B. dict
C. set
D. bytearray

答案 A. list；B. dict；C. set；D. bytearray

解析 题目）

32 • 不定项选择

以下哪些操作会创建列表的副本？

A. lst[:]
B. lst.copy()
C. list(lst)
D. lst

答案 A. lst[:]；B. lst.copy()；C. list(lst)；D. lst

解析 目）

33 • 不定项选择

关于元组包含可变对象，以下说法正确的有：

A. 元组本身不可变
B. 元组中的可变对象可以被修改
C. 这样的元组不能作为字典的键
D. 元组的哈希值会改变

答案 A. 元组本身不可变；B. 元组中的可变对象可以被修改；C. 这样的元组不能作为字典的键

解析

34 • 不定项选择

以下哪些方法会修改原列表？

A. append()
B. extend()
C. insert()
D. remove()

答案 A. append()；B. extend()；C. insert()；D. remove()

解析

35 • 不定项选择

关于字符串的不可变性，以下说法正确的有：

A. 字符串操作总是返回新字符串
B. 字符串不能被修改
C. 字符串拼接效率较低
D. 字符串可以作为字典的键

答案 A. 字符串操作总是返回新字符串；B. 字符串不能被修改；C. 字符串拼接效率较低；D. 字符串可以作为字典的键

解析

36 • 不定项选择

以下哪些是避免可变默认参数问题的方法？

A. 使用None作为默认值
B. 使用元组代替列表
C. 在函数内部创建新对象
D. 使用不可变对象

答案 A. 使用None作为默认值；B. 使用元组代替列表；C. 在函数内部创建新对象；D. 使用不可变对象

解析

37 • 不定项选择

关于深拷贝和浅拷贝的应用场景，以下说法正确的有：

A. 简单对象用浅拷贝
B. 嵌套对象用深拷贝
C. 深拷贝更安全但更慢
D. 浅拷贝更快但可能有副作用

答案 A. 简单对象用浅拷贝；B. 嵌套对象用深拷贝；C. 深拷贝更安全但更慢；D. 浅拷贝更快但可能有副作用

解析

38 • 不定项选择

以下哪些操作不会修改原字典？

A. d.get(key)
B. d.keys()
C. d.pop(key)
D. d.items()

答案 A. d.get(key)；B. d.keys()；D. d.items()

解析

39 • 不定项选择

关于列表的切片赋值，以下说法正确的有：

A. 可以改变列表长度
B. 会修改原列表
C. 可以删除元素
D. 可以插入元素

答案 A. 可以改变列表长度；B. 会修改原列表；C. 可以删除元素；D. 可以插入元素

解析

40 • 不定项选择

关于函数参数的可变性陷阱，以下说法正确的有：

A. 传递可变对象时要小心
B. 函数内部修改会影响外部
C. 可以通过传递副本避免
D. 不可变对象没有这个问题

答案 A. 传递可变对象时要小心；B. 函数内部修改会影响外部；C. 可以通过传递副本避免；D. 不可变对象没有这个问题

解析

41 • 不定项选择

以下哪些是Pandas中创建副本的方法？

A. df.copy()

B. df.copy(deep=True)
C. df[:]
D. df.loc[:, :]

答案 A. df.copy()；B. df.copy(deep=True)

解析

42 • 不定项选择

关于集合的元素要求，以下说法正确的有：

A. 元素必须可哈希
B. 元素必须不可变
C. 不能包含列表
D. 可以包含元组

答案 A. 元素必须可哈希；B. 元素必须不可变；C. 不能包含列表；D. 可以包含元组

解析

43 • 不定项选择

关于列表推导式和可变性，以下说法正确的有：

A. 列表推导式创建新列表
B. 不会修改原数据
C. 可以包含可变对象
D. 比for循环更安全

答案 A. 列表推导式创建新列表；B. 不会修改原数据；C. 可以包含可变对象

解析

44 • 不定项选择

以下哪些操作会改变对象的id？

A. 列表的append()
B. 字符串的拼接
C. 整数的加法
D. 元组的拼接

答案 B. 字符串的拼接；C. 整数的加法；D. 元组的拼接

解析

45 • 不定项选择

关于Pandas的链式索引警告，以下说法正确的有：

A. 可能产生副本
B. 修改可能不生效
C. 应该使用loc或iloc
D. 可以忽略警告

答案 A. 可能产生副本；B. 修改可能不生效；C. 应该使用loc或iloc

解析

三、填空题

46 • 填空 / 简答

列表的 _____ 方法会修改原列表，而 _____ 函数返回新列表。

答案 sort() ， sorted()

解析 列表排序的两种方式：

- sort() ：就地排序
  就地排序，修改原列表，返回None
- sorted() ：返回新排序列表
  新排序列表，原列表不变

```
lst = [3, 1, 2]
lst.sort()
# 修改lst，lst=[1,2,3]
new_lst = sorted([3,1,2])
# 返回新列表[1,2,3]
```

47 • 填空 / 简答

可变对象作为默认参数时，应该使用 _____ 作为默认值，然后在函数内部判断并创建 _____。

答案 None ， 新对象

解析 新对象

就地修改方法通常返回None。

```
lst = [3, 1, 2]
result = lst.sort()
# 就地排序
print(result)
# None
print(lst)
# [1, 2, 3]
lst2 = [3, 1, 2]
result2 = sorted(lst2)
# 返回新列表
print(result2)
# [1, 2, 3]
print(lst2)
# [3, 1, 2] - 未变
```

48 • 填空 / 简答

元组的不可变性体现在 _____ 不可变，但如果元组包含 _____ 对象，该对象的内容仍可修改。

答案 元素引用

解析 元素引用 ， 可变
可变

幫析：浅拷贝的元素共享问题。

```
import copy
original = [1, [2, 3], 4]
shallow = copy.copy(original)
shallow[1].append(999)
print(original)
```

```
# [1, [2, 3, 999], 4] - 受影响
```

49 • 填空 / 简答

在Pandas中，使用 _____ 参数可以确保操作返回副本而不是视图，使用 _____ 方法可以显式创建副本。

答案 copy=True ， copy()

解析 幂析：Pandas复制数据的方式。

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2, 3]})
# copy方法
df_copy = df.copy()
df_copy.loc[0, 'A'] = 999
print(df)
# 原数据未变
```

50 • 填空 / 简答

字符串是 _____ 对象，所有字符串方法都返回 _____ 而不修改原字符串。

答案 不可变

解析 不可变 ， 新字符串
新字符串
幂析：字符串不可变性。

```
s = "hello"
s_upper = s.upper()
# 返回新字符串
print(s)
# hello - 未变
print(s_upper)
# HELLO
```

四、程序代码阅读题

51 • 填空 / 简答

阅读以下代码，回答问题：

```
class DataHolder:
    def __init__(self, data=[]):
        self.data = data
    def add(self, item):
        self.data.append(item)
obj1 = DataHolder()
obj2 = DataHolder()
obj1.add(1)
obj2.add(2)
print(obj1.data)
print(obj2.data)
```

问题：

• (1) 两个print语句的输出分别是什么？

- (2) 为什么会出现这种情况？
- (3) 如何修改类定义避免这个问题？

答案 • (1) 输出：

```
[1, 2]
```

- (2) items是类属性，所有实例共享同一个列表对象
- (3) 修改方法：

```
class DataHolder:
    def __init__(self, data=None):
        self.data = data if data is not None else []
```

解析

52 • 填空 / 简答

阅读以下代码，回答问题：

```
def process_list(lst, operation='append', value=None):
    if operation == 'append':
        lst.append(value)
    elif operation == 'new':
        lst = [value]
    elif operation == 'extend':
        lst.extend(value)
    return lst
data = [1, 2, 3]
result1 = process_list(data, 'append', 4)
result2 = process_list(data, 'new', 10)
result3 = process_list(data, 'extend', [5, 6])
print(data)
print(result2)
```

问题：

- (1) 两个print语句的输出分别是什么？
- (2) 为什么'new'操作不影响原列表？
- (3) 哪些操作修改了原列表？

答案 • (1) 输出：

```
[1, 2, 3, 4, 5, 6]
[10]
```

- (2) 'new'操作中 lst = [value] 创建新对象，不影响原列表
- (3) 'append'和'extend'操作修改了原列表

解析

53 • 填空 / 简答

阅读以下代码，回答问题：

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
# 方法1
subset1 = df[df['A'] > 1]
subset1['C'] = [10, 20]
# 方法2
```

```
subset2 = df[df['A'] > 1].copy()
subset2['D'] = [30, 40]
print('C' in df.columns)
print('D' in df.columns)
```

问题：

- (1) 两个print语句的输出分别是什么？
- (2) 为什么需要使用copy()？
- (3) 什么是SettingWithCopyWarning?

答案 使用

- (1) 输出：

```
False
```

或可能是： True False （取决于Pandas版本）

- (2) 使用copy()创建独立副本，修改副本不影响原DataFrame
- (3) SettingWithCopyWarning是Pandas在链式索引可能修改副本而非原数据时发出的警告

解析

54 • 填空 / 简答

阅读以下代码，回答问题：

```
def create_matrix(rows, cols, default=0):
    return [[default] * cols for _ in range(rows)]
def create_matrix_wrong(rows, cols, default=0):
    return [[default] * cols] * rows
matrix1 = create_matrix(3, 3, 0)
matrix2 = create_matrix_wrong(3, 3, 0)
matrix1[0][0] = 1
matrix2[0][0] = 1
print(matrix1)
print(matrix2)
```

问题：

- (1) 两个print语句的输出分别是什么？
- (2) 为什么matrix2的所有行都被修改了？
- (3) 正确的矩阵创建方法是什么？

答案 • (1) 输出：

```
[[1, 0, 0], [0, 0, 0], [0, 0, 0]]
[[1, 0, 0], [1, 0, 0], [1, 0, 0]]
```

- (2) [[default] * cols] * rows 创建的是同一个列表的多个引用
- (3) 使用列表推导式： [[default] * cols for _ in range(rows)]

解析

55 • 填空 / 简答

阅读以下代码，回答问题：

```
def update_config(config, updates):
    for key, value in updates.items():
        if isinstance(value, dict) and key in config:
            update_config(config[key], value)
        else:
```

```
            config[key] = value
    return config
original_config = {
    'database': {'host': 'localhost', 'port': 5432},
    'debug': True
}
new_config = update_config(original_config, {
    'database': {'port': 3306},
    'debug': False
})
print(original_config is new_config)
print(original_config['database']['port'])
```

问题：

- (1) 两个print语句的输出分别是什么？
- (2) 这个函数有什么问题？
- (3) 如何修改使其不影响原配置？

答案 • (1) 输出：

```
True
3306
```

- (2) 函数直接修改了传入的config对象
- (3) 修改方法：

```
def update_config(config, updates):
    import copy
    new_config = copy.deepcopy(config)
    for key, value in updates.items():
        if isinstance(value, dict) and key in new_config:
            new_config[key] = update_config(new_config[key], value)
        else:
            new_config[key] = value
    return new_config
```

解析

五、程序设计题

56 • 填空 / 简答

实现一个不可变列表类 ImmutableList ：

1. 基本功能

基本功能：

- 支持索引访问
- 支持切片
- 支持迭代
- 支持len()

2. 不可变性保证

不可变性保证：

- 任何修改操作返回新对象

- 原对象保持不变
- 实现 append() 、 extend() 等方法（返回新对象）

3. 性能优化

性能优化：

- 使用元组内部存储
- 实现结构共享（避免完全复制）
- 提供 to_list() 转换方法

4. 额外功能

额外功能：

- 实现 __hash__() 使其可作为字典键
- 实现 __add__() 支持+操作
- 提供 map() 、 filter() 等函数式方法

答案

```
class ImmutableList:
    """
    不可变列表实现
    设计思路：
    1. 内部使用元组存储数据，保证不可变性
    2. 所有修改操作返回新的ImmutableList对象
    3. 实现__hash__使其可作为字典键
    4. 提供函数式编程接口
    """
    def __init__(self, items=()):
        # 使用元组作为内部存储，确保不可变性
        if isinstance(items, tuple):
            self._data = items
        else:
            self._data = tuple(items)
    # === 基本访问方法 ===
    def __getitem__(self, index):
        """支持索引和切片访问"""
        if isinstance(index, slice):
            return ImmutableList(self._data[index])
        return self._data[index]
    def __len__(self):
        """返回长度"""
        return len(self._data)
    def __iter__(self):
        """支持迭代"""
        return iter(self._data)
    def __repr__(self):
        """字符串表示"""
        return f"ImmutableList({list(self._data)})"
    # === 不可变性保证 ===
    def __hash__(self):
    """实现哈希，使其可作为字典键"""
    return hash(self._data)
def __eq__(self, other):
```

```
    """相等性比较"""
    if isinstance(other, ImmutableList):
        return self._data == other._data
    return False
# === 修改操作（返回新对象）===
def append(self, item):
    """添加元素，返回新对象"""
    return ImmutableList(self._data + (item,))
def extend(self, items):
    """扩展列表，返回新对象"""
    return ImmutableList(self._data + tuple(items))
def insert(self, index, item):
    """插入元素，返回新对象"""
    return ImmutableList(self._data[:index] + (item,) + self._data[index:])
def remove(self, item):
    """移除元素，返回新对象"""
    data_list = list(self._data)
    data_list.remove(item)
    return ImmutableList(data_list)
def pop(self, index=-1):
    """弹出元素，返回(新列表, 弹出的元素)"""
    data_list = list(self._data)
    popped = data_list.pop(index)
    return ImmutableList(data_list), popped
# === 运算符重载 ===
def __add__(self, other):
    """支持+操作"""
    if isinstance(other, ImmutableList):
        return ImmutableList(self._data + other._data)
    return ImmutableList(self._data + tuple(other))
    def __mul__(self, n):
        """支持*操作"""
        return ImmutableList(self._data * n)
    # === 函数式方法 ===
    def map(self, func):
        """映射操作"""
        return ImmutableList(func(item) for item in self._data)
    def filter(self, predicate):
        """过滤操作"""
        return ImmutableList(item for item in self._data if predicate(item))
    def reduce(self, func, initial=None):
        """归约操作"""
        from functools import reduce as functools_reduce
        if initial is None:
            return functools_reduce(func, self._data)
        return functools_reduce(func, self._data, initial)
    # === 转换方法 ===
    def to_list(self):
```

```
        """转换为普通列表"""
        return list(self._data)
    def to_tuple(self):
        """转换为元组"""
        return self._data
# === 使用示例 ===
print("=== ImmutableList 示例 ===\n")
# 创建不可变列表
lst = ImmutableList([1, 2, 3])
print(f"原列表: {lst}")
# 添加元素（返回新对象）
lst2 = lst.append(4)
print(f"添加后: {lst2}")
print(f"原列表不变: {lst}")
# 扩展列表
lst3 = lst2.extend([5, 6])
print(f"扩展后: {lst3}")
# 函数式操作
doubled = lst.map(lambda x: x * 2)
print(f"\nMap操作: {doubled}")
evens = lst3.filter(lambda x: x % 2 == 0)
print(f"Filter操作: {evens}")
total = lst3.reduce(lambda a, b: a + b)
print(f"Reduce操作: {total}")
# 作为字典键
d = {lst: "value1", lst2: "value2"}
print(f"\n作为字典键: {d}")
# 链式操作
result = (ImmutableList([1, 2, 3])
    .append(4)
    .extend([5, 6])
    .map(lambda x: x ** 2)
    .filter(lambda x: x > 10))
print(f"链式操作结果: {result}")
```

解析

57 • 填空 / 简答

实现参数验证和保护装饰器：

1. @validate_types：

- 验证参数类型
- 支持类型注解
- 类型不匹配时抛出异常

2. @protect_mutable：

- 自动复制可变参数
- 防止函数修改外部对象
- 可配置深拷贝或浅拷贝

3. @freeze_return：

- 将返回的可变对象转为不可变
- 列表转元组
- 字典转frozendict

4. 综合示例

综合示例：

- 组合使用多个装饰器
- 处理复杂嵌套结构
- 提供详细的错误信息

答案

```
import copy
from functools import wraps
import inspect
from typing import get_type_hints
def validate_types(func):
    """
    验证参数类型装饰器
    功能说明：
    1. 基于函数的类型注解验证参数类型
    2. 类型不匹配时抛出TypeError
    3. 保留原函数的元数据
    """
    @wraps(func)
    def wrapper(*args, **kwargs):
        # 获取函数签名和类型注解
        sig = inspect.signature(func)
        bound = sig.bind(*args, **kwargs)
        bound.apply_defaults()
        # 获取类型提示
        hints = get_type_hints(func)
        # 验证每个参数
        for param_name, param_value in bound.arguments.items():
            if param_name in hints:
                expected_type = hints[param_name]
                if not isinstance(param_value, expected_type):
                raise TypeError(
                f"参数 '{param_name}' 期望类型{expected_type.__name__}, "
                f"但得到{type(param_value).__name__}"
                )
        return func(*args, **kwargs)
    return wrapper
def protect_mutable(deep=False):
    """
    保护可变参数装饰器
    参数说明：
    - deep: True使用深拷贝，False使用浅拷贝
    功能说明：
    1. 自动复制可变参数
    2. 防止函数修改外部对象
```

```
    3. 可配置拷贝深度
    """
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            # 复制可变参数
            new_args = []
            for arg in args:
                if isinstance(arg, (list, dict, set)):
                if deep:
                new_args.append(copy.deepcopy(arg))
                else:
                new_args.append(copy.copy(arg))
                else:
                new_args.append(arg)
            new_kwargs = {}
            for key, value in kwargs.items():
                if isinstance(value, (list, dict, set)):
                if deep:
                new_kwargs[key] = copy.deepcopy(value)
                else:
                new_kwargs[key] = copy.copy(value)
                else:
                new_kwargs[key] = value
            return func(*new_args, **new_kwargs)
        return wrapper
    return decorator
def freeze_return(func):
    """
    冻结返回值装饰器
    功能说明：
    1. 将返回的可变对象转为不可变
    2. list → tuple
    3. dict → frozendict (使用元组模拟)
    4. set → frozenset
    """
    @wraps(func)
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        return _make_immutable(result)
    return wrapper
def _make_immutable(obj):
    """递归将对象转为不可变"""
    if isinstance(obj, list):
        return tuple(_make_immutable(item) for item in obj)
    elif isinstance(obj, dict):
        # 使用元组模拟frozendict
        return tuple((k, _make_immutable(v)) for k, v in sorted(obj.items()))
```

```
    elif isinstance(obj, set):
        return frozenset(_make_immutable(item) for item in obj)
    else:
        return obj
# === 使用示例 ===
print("=== 参数验证装饰器示例 ===\n")
# 示例1：类型验证
@validate_types
def greet(name: str, age: int) -> str:
    return f"Hello {name}, you are {age} years old"
try:
    print(greet("Alice", 25))
    print(greet("Bob", "30"))
# 类型错误
except TypeError as e:
    print(f"类型错误: {e}")
# 示例2：保护可变参数
@protect_mutable(deep=True)
def modify_list(lst):
    """这个函数修改列表，但不会影响外部"""
    lst.append(999)
    return lst
original = [1, 2, 3]
result = modify_list(original)
print(f"\n原列表: {original}")
# [1, 2, 3]
print(f"返回值: {result}")
# [1, 2, 3, 999]
# 示例3：冻结返回值
@freeze_return
def create_config():
    """返回配置字典，但会被冻结"""
    return {
        'name': 'app',
        'settings': {'debug': True, 'port': 8000},
        'features': ['auth', 'api']
    }
config = create_config()
print(f"\n冻结的配置: {config}")
print(f"类型: {type(config)}")
# 示例4：组合使用装饰器
@freeze_return
@protect_mutable()
@validate_types
def process_data(data: list, multiplier: int) -> list:
    """处理数据：验证类型、保护参数、冻结返回值"""
    return [x * multiplier for x in data]
data = [1, 2, 3]
```

```
result = process_data(data, 2)
print(f"\n原数据: {data}")
print(f"处理结果: {result}")
print(f"结果类型: {type(result)}")
```

解析

58 • 填空 / 简答

实现考虑可变性的缓存系统：

1. CacheKey类：

- 将可变对象转为可哈希的键
- 支持列表、字典等
- 处理嵌套结构

2. SmartCache类：

- 使用不可变键存储
- 自动检测参数变化
- 支持TTL过期

3. 缓存策略

缓存策略：

- LRU淘汰策略
- 基于内存限制的淘汰
- 缓存统计和监控

4. 应用示例

应用示例：

- 装饰器形式使用
- 函数结果缓存
- 处理可变参数的函数

答案

```
from collections import OrderedDict
import time
import hashlib
import pickle
class CacheKey:
    """
    将可变对象转为可哈希的缓存键
    设计思路：
    1. 将任意对象序列化为字符串
    2. 计算哈希值作为缓存键
    3. 处理嵌套结构和循环引用
    """
    @staticmethod
    def make_key(*args, **kwargs):
        """创建缓存键"""
        try:
            # 尝试直接哈希
            key_data = (args, tuple(sorted(kwargs.items())))
            return hash(key_data)
        except TypeError:
            # 对于不可哈希的对象，使用pickle序列化
```

```
            serialized = pickle.dumps((args, kwargs))
            return hashlib.md5(serialized).hexdigest()
class SmartCache:
    """
    智能缓存系统
    功能特性：
    1. 自动处理可变参数
    2. 支持TTL过期
    3. LRU淘汰策略
    4. 缓存统计
    """
def __init__(self, maxsize=128, ttl=None):
    """
    参数说明：
    - maxsize: 最大缓存条目数
    - ttl: 过期时间（秒），None表示不过期
    """
    self.maxsize = maxsize
    self.ttl = ttl
    self.cache = OrderedDict()
    self.timestamps = {}
    self.hits = 0
    self.misses = 0
def get(self, *args, **kwargs):
    """获取缓存值"""
    key = CacheKey.make_key(*args, **kwargs)
    # 检查是否存在
    if key not in self.cache:
        self.misses += 1
        return None
    # 检查是否过期
    if self.ttl is not None:
        if time.time() - self.timestamps[key] > self.ttl:
            del self.cache[key]
            del self.timestamps[key]
            self.misses += 1
            return None
    # 命中：移到末尾（LRU）
    self.cache.move_to_end(key)
    self.hits += 1
    return self.cache[key]
def put(self, value, *args, **kwargs):
    """存入缓存"""
    key = CacheKey.make_key(*args, **kwargs)
        # 如果已存在，更新并移到末尾
        if key in self.cache:
            self.cache.move_to_end(key)
        self.cache[key] = value
```

```
        self.timestamps[key] = time.time()
        # 检查大小限制
        if len(self.cache) > self.maxsize:
            # 删除最旧的条目
            oldest_key = next(iter(self.cache))
            del self.cache[oldest_key]
            del self.timestamps[oldest_key]
    def clear(self):
        """清空缓存"""
        self.cache.clear()
        self.timestamps.clear()
        self.hits = 0
        self.misses = 0
    def stats(self):
        """获取统计信息"""
        total = self.hits + self.misses
        hit_rate = self.hits / total if total > 0 else 0
        return {
            'hits': self.hits,
            'misses': self.misses,
            'hit_rate': hit_rate,
            'size': len(self.cache),
            'maxsize': self.maxsize
        }
def cached(cache_instance):
    """
    缓存装饰器
    使用方法:
    @cached(my_cache)
    def expensive_function(x, y):
        ...
    """
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            # 尝试从缓存获取
            cached_value = cache_instance.get(*args, **kwargs)
            if cached_value is not None:
                return cached_value
            # 计算结果
            result = func(*args, **kwargs)
            # 存入缓存
            cache_instance.put(result, *args, **kwargs)
            return result
        wrapper.cache = cache_instance
        return wrapper
    return decorator
# === 使用示例 ===
```

```
print("=== 智能缓存系统示例 ===\n")
# 创建缓存实例
cache = SmartCache(maxsize=100, ttl=5)
# 示例1：处理可变参数
@cached(cache)
def process_list(data, multiplier):
    """耗时操作"""
    print(f" 计算: {data} * {multiplier}")
    time.sleep(0.1)
    return [x * multiplier for x in data]
print("第1次调用（未命中）:")
result1 = process_list([1, 2, 3], 2)
print(f"结果: {result1}")
print("\n第2次调用（命中）:")
result2 = process_list([1, 2, 3], 2)
print(f"结果: {result2}")
print("\n第3次调用（不同参数，未命中）:")
result3 = process_list([1, 2, 3], 3)
print(f"结果: {result3}")
# 统计信息
print(f"\n缓存统计: {cache.stats()}")
# 示例2：TTL过期测试
print("\n=== TTL过期测试 ===")
cache_ttl = SmartCache(maxsize=10, ttl=2)
@cached(cache_ttl)
def slow_function(x):
    print(f" 计算{x}")
    return x ** 2
print("首次调用:")
print(slow_function(5))
print("\n立即再次调用（缓存命中）:")
print(slow_function(5))
print("\n等待3秒后调用（缓存过期）:")
time.sleep(3)
print(slow_function(5))
print(f"\n统计: {cache_ttl.stats()}")
```

评分说明：

- 单选题：每题1分，共30分
- 多选题：每题2分，选对但不全得1分，有错选不得分，共30分
- 填空题：每空1分，共10分
- 代码阅读题：每题3分，共15分
- 程序设计题：5选3，每题5分，共15分
- 总分：

总分：100分

解析

59 • 填空 / 简答

实现数据对象的版本控制系统：
1. VersionedData类：
• 记录所有修改历史
• 每次修改创建新版本
• 支持回滚到任意版本
2. 差异追踪
差异追踪：
• 记录每次修改的差异
• 最小化存储空间
• 支持差异查看
3. 分支管理
分支管理：
• 支持创建分支
• 支持合并分支
• 处理冲突
4. 持久化
持久化：
• 保存版本历史到文件
• 使用pickle或JSON
• 支持增量保存

答案 （原卷为 5 选 3，未提供本题参考答案；考试时任选 3 题作答即可）

解析

60 • 填空 / 简答

实现Pandas的安全操作包装器：
1. SafeDataFrame类：
• 继承DataFrame
• 所有修改操作自动创建副本
• 提供明确的inplace控制
2. 链式操作支持
链式操作支持：
• 支持方法链
• 每步返回新对象
• 保持操作历史
3. 视图检测
视图检测：
• 自动检测视图操作
• 警告潜在的副作用
• 提供安全的替代方法
4. 性能优化
性能优化：
• 延迟复制（写时复制）
• 智能判断何时需要复制
• 提供性能分析工具

答案 （原卷为 5 选 3，未提供本题参考答案；考试时任选 3 题作答即可）

解析